

Guida d'uso

Riprodurre la pipeline di rumore sul dimero in locale

Pacchetto riproducibile — Tesi triennale, Samuele Schiavi

Relatore: Prof. Alessandro Chiesa

Questo pacchetto contiene tutto il codice necessario per riprodurre, da zero, ogni simulazione, ogni numero e ogni figura usati nei documenti sulla Parte 2 (rumore quantistico) del dimero — senza dipendere da nessun file già calcolato: tutto si rigenera in locale. Ogni modulo e ogni funzione del pacchetto sono commentati esplicitando sia il loro ruolo all'interno del proprio file sia il loro ruolo nell'insieme della pipeline — questa guida ne è la mappa, non la ripetizione.

Indice

1	Struttura del pacchetto	1
1.1	I 16 moduli di <code>codice/</code>	1
2	Prerequisiti	2
3	Il modo più semplice: un solo comando	2
4	Passo per passo, con la mappa esatta delle dipendenze	3
4.1	Onda 1 — nessuna dipendenza	3
4.2	Onda 2 — dipende SOLO da <code>01_ (ground_state_test2.npz)</code>	4
4.3	Onda 2' — dipende SOLO da <code>03_</code>	5
4.4	Onda 3 — dipende da <code>01_ E 02_</code>	5
4.5	Il grafo delle dipendenze, in breve	6
4.6	Tutti i comandi, in un unico blocco (nell'ordine di <code>esegui_tutto.py</code>)	6
5	Come sapere se un numero è quello giusto	6
6	Due insidie già trovate e corrette	7
7	Cosa NON è incluso in questo pacchetto	7

1 Struttura del pacchetto

<code>codice/</code>	16 moduli Python (mattoni + pipeline + estensioni)
<code>dati/</code>	dati intermedi (<code>.npz</code>) generati dagli script 01–03
<code>figure/</code>	le 15 figure finali, in PDF e PNG
<code>genera_figure/</code>	13 script, uno o più per ciascuna figura
<code>01_genera_dati_vqe_ideale.py</code>	primo script da eseguire
<code>02_genera_dati_vqe_noise_aware.py</code>	secondo script
<code>03_genera_dati_secondo_punto.py</code>	terzo script (punto di lavoro alternativo)
<code>esegui_tutto.py</code>	esegue tutto in un solo comando

I 16 moduli di `codice/`

File	Cosa fa
dimer_exact.py	Hamiltoniana e diagonalizzazione esatta — il “mattone zero”, sorgente unica per tutto il pacchetto
vqe_test2.py	ansatz PMA-2q.3 e ottimizzazione VQE ideale (nessun rumore)
circuito_correlazioni_dimero.py	circuito di Hadamard test per i correlatori, a rumore nullo (versione con supporto preparazione VQE reale)
noise_model_dimero.py	costruzione del NoiseModel (canale depolarizzante + readout, calibrazione ibm_torino)
vqe_dm_rumoroso_dimero.py	VQE con termine DM sotto rumore (riuso dei parametri ideali)
trotter_rumoroso_dimero.py	quantum simulation (Trotter) sotto rumore
correlatori_rumorosi_dimero.py	correlatori dinamici sotto rumore (readout simmetrico)
scan_parametri_rumore_dimero.py	scan di N^* al variare di $\varepsilon_{1q}, \varepsilon_{2q}$
vqe_noise_aware_dimero.py	estensione: ottimizzazione ripetuta del VQE sotto rumore
verifica_passo5_noise_aware.py	estensione: verifica su tutta la griglia dello scan
correlatori_readout_asimmetrico.py	estensione: errore di lettura asimmetrico
stile.py	stile grafico condiviso da tutti gli script di figura
validate_correlatori_rumorosi_dimero.py	controllo obbligatorio: limite rumore nullo, conteggio gate, formula readout
validate_scan_parametri_rumore_dimero.py	controllo obbligatorio: nessuna regressione, monotonia, invarianza da p_{readout}
validate_vqe_noise_aware_dimero.py	controllo obbligatorio: 8 verifiche sull'estensione VQE noise-aware
validate_correlatori_readout_asimmetrico.py	controllo obbligatorio: 4 verifiche sull'estensione readout asimmetrico

2 Prerequisiti

Testato con questa combinazione esatta di versioni (altre versioni recenti di Qiskit 2.x dovrebbero funzionare, ma non sono state verificate):

Python	3.12.3
qiskit	2.5.2
qiskit-aer	0.17.2
numpy	2.4.4
scipy	1.17.1
matplotlib	3.10.8

Installazione consigliata (ambiente virtuale):

```
python3 -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install qiskit qiskit-aer numpy scipy matplotlib
```

Nessuna installazione LaTeX richiesta

Le figure sono generate direttamente in PDF/PNG da matplotlib, senza passare da un motore di rendering matematico esterno (`mathtext.fontset: cm` di matplotlib, autosufficiente).

3 Il modo più semplice: un solo comando

```
cd pacchetto_reproducibile
python3 esegui_tutto.py
```

Esegue in ordine: generazione dati (3 script), validazioni (4 script), tutte le 15 figure (13 script). Si ferma al primo errore, indicando esattamente quale fase e comando ha fallito.

Tempo stimato

10–20 minuti. Le fasi più lente sono gli script di figura che ripetono ottimizzazioni VQE a più valori di rumore.

Se tutto va a buon fine, l'ultima riga stampata è:

TUTTO COMPLETATO. 15 figure in figure/, dati in dati/.

4 Passo per passo, con la mappa esatta delle dipendenze

Diciassette script in tutto (3 dati, 4 validazioni, 13 di figura). Invece di un elenco lineare, questa sezione li organizza per **onde di dipendenza reale** — verificate leggendo ogni `np.load/np.savez` nel codice, non assunte. Alcune dipendenze sono meno ovvie del previsto: `03_` non ha bisogno di `01_/02_` (calcola tutto da zero per il secondo punto), e la figura del secondo punto ha bisogno solo di `03_`, non degli altri due script dati.

Onda 1 — nessuna dipendenza

01_genera_dati_vqe_ideale.py

Ingresso: nessuno.

Uscita: `dati/ground_state_test2.npz` — `b,J,D`, `E0_exact`, `psi0_exact`, `vqe_params` (3 parametri PMA-2q.3), `fidelity`, `E_vqe`.

Chi lo consuma: quasi tutto il resto del pacchetto (Onda 2) — è il prerequisito più condiviso. Fanno eccezione solo `03_` e le figure che dipendono esclusivamente da `03_` (vedi sotto).

Valore atteso: energia ≈ -3.57321145 , fedeltà ≈ 1.0000000000 .

Comando (dalla radice del pacchetto):

`python3 01_genera_dati_vqe_ideale.py`

03_genera_dati_secondo_punto.py

Ingresso: nessuno — NON legge `ground_state_test2.npz` né `vqe_noise_aware_result.npz`: ottimizza da zero sia il VQE ideale sia quello noise-aware, al secondo punto ($b/J = -0.18$, $D/J = 1$).

Uscita: `dati/ground_state_punto2.npz` (analogo del file di `01_` ma per il secondo punto) e `dati/vqe_noise_aware_punto2.npz` (analogo del file di `02_`).

Chi lo consuma: solo `genera_fig_secondo_punto.py`.

Valori attesi: $\Delta E \approx -2.56 \times 10^{-8}$, N^* Trotter $8 \rightarrow 8$, N^* correlatore $3 \rightarrow 3$.

Comando (dalla radice del pacchetto):

`python3 03_genera_dati_secondo_punto.py`

genera_fig_modello_rumore.py

Ingresso: nessuno — figura puramente concettuale (contrazione del vettore di Bloch), nessun dato del progetto richiesto.

Uscita: `figure/fig_depolarizzazione_bloch.pdf`.

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto):

`python3 genera_figure/genera_fig_modello_rumore.py`

Onda 2 — dipende SOLO da 01_ (ground_state_test2.npz)

02_genera_dati_vqe_noise_aware.py

Ingresso: dati/ground_state_test2.npz (vqe_params, psi0_exact).**Uscita:** dati/vqe_noise_aware_result.npz — x_noise_aware, E_noise_aware, F_noise_aware, E_ideale_su_rumore, F_ideale_su_rumore, cnot_invarianza, dati del confronto sul correlatore (N_grid_corr, vals_corr_ideale/na, N_star_corr_ideale/na).**Chi lo consuma:** le 4 figure dell'Onda 3 (uniche che hanno bisogno anche di questo file, oltre a 01_).**Valori attesi:** $\Delta E \approx -1.16 \times 10^{-7}$, $\Delta F \approx +1.76 \times 10^{-8}$, N^* del correlatore $5 \rightarrow 5$.**Comando** (dalla radice del pacchetto, dopo 01_):

python3 02_genera_dati_vqe_noise_aware.py

I 4 controlli di validazione Tutti e quattro leggono **solo** ground_state_test2.npz (copiato in codice/) — nessuno tocca l'output di 02_ o 03_, nemmeno validate_vqe_noise_aware_dimero.py (che pure valida l'estensione noise-aware): ricalcola tutto da zero invece di leggere una cache. Nessuno produce file, solo un resoconto a schermo (AssertionError se un controllo fallisce); nessuno è consumato da altri script (sono tutti controlli terminali). Prerequisito comune per tutti e quattro, da fare una sola volta:

```
cp dati/ground_state_test2.npz codice/
cd codice
```

validate_correlatori_rumorosi_dimero.py

Comando: python3 validate_correlatori_rumorosi_dimero.py

validate_scan_parametri_rumore_dimero.py

Comando: python3 validate_scan_parametri_rumore_dimero.py

validate_vqe_noise_aware_dimero.py

Comando: python3 validate_vqe_noise_aware_dimero.py

validate_correlatori_readout_asimmetrico.py

Comando: python3 validate_correlatori_readout_asimmetrico.py

Dopo l'ultimo controllo, torna alla radice: cd ..

7 figure che dipendono solo da 01_ Ingresso comune per tutte e sette: dati/ground_state_test2.npz soltanto. **Uscita:** la rispettiva figura in figure/. Nessuna è consumata da altri script del pacchetto (punti di arrivo). Comandi, tutti dalla **radice** del pacchetto:

genera_fig_vqe_dm_rumoroso.py → fig_passo2_scan_eps2q

python3 genera_figure/genera_fig_vqe_dm_rumoroso.py

genera_fig_trotter_rumoroso.py → fig_passo3_F_vs_N, fig_passo3_zoom_minimo

python3 genera_figure/genera_fig_trotter_rumoroso.py

genera_fig_correlatori_rumorosi.py → fig_passo4_correlatore_vs_N

python3 genera_figure/genera_fig_correlatori_rumorosi.py

```
genera_fig_scan_parametri.py → fig_passo5_Nstar_vs_eps
```

```
python3 genera_figure/genera_fig_scan_parametri.py
```

```
genera_fig_trappola_transpilazione.py → fig_bug_conteggio_gate
```

```
python3 genera_figure/genera_fig_trappola_transpilazione.py
```

```
genera_fig_scan_eps2q_confronto.py → fig_scan_eps2q_confronto
```

Confronta ideale e noise-aware, ma ottimizza il noise-aware fresco ad ogni valore di ε_{2q} invece di leggere 02_ — per questo resta in questa onda e non nella successiva.

```
python3 genera_figure/genera_fig_scan_eps2q_confronto.py
```

```
genera_fig_readout_asimmetrico.py → fig_correlatore_N_asimmetrico,
fig_Nstar_vs_rapporto
```

```
python3 genera_figure/genera_fig_readout_asimmetrico.py
```

Onda 2' — dipende SOLO da 03_

```
genera_fig_secondo_punto.py
```

Ingresso: dati/ground_state_punto2.npz e dati/vqe_noise_aware_punto2.npz — NON tocca ground_state_test2.npz né vqe_noise_aware_result.npz: è indipendente da 01_ e 02_, dipende solo da 03_.

Uscita: figure/fig_secondo_punto_confronto.pdf.

Chi la consuma: nessuno (punto di arrivo).

Comando (dalla radice del pacchetto, dopo 03_):

```
python3 genera_figure/genera_fig_secondo_punto.py
```

Onda 3 — dipende da 01_ E 02_

4 figure che confrontano ideale e noise-aware Ingresso comune per tutte e quattro: dati/ground_state_test2.npz e dati/vqe_noise_aware_result.npz — le uniche figure di questo pacchetto che hanno davvero bisogno dell'output di 02_. Uscita: la rispettiva figura in figure/. Nessuna consumata da altri script. Comandi, dalla **radice** del pacchetto:

```
genera_fig_F_vs_N_confronto.py → fig_F_vs_N_confronto
```

```
python3 genera_figure/genera_fig_F_vs_N_confronto.py
```

```
genera_fig_correlatore_confronto.py → fig_correlatore_noise_aware_confronto
```

```
python3 genera_figure/genera_fig_correlatore_confronto.py
```

```
genera_fig_robustezza_multistart.py → fig_robustezza_multistart
```

```
python3 genera_figure/genera_fig_robustezza_multistart.py
```

```
genera_fig_griglia_completa.py → fig_passo5_griglia_completa
```

```
python3 genera_figure/genera_fig_griglia_completa.py
```

Il grafo delle dipendenze, in breve

Tre catene indipendenti, non una sola sequenza

- 01_ → (4 validate + 7 figure dirette) → 02_ → (4 figure di confronto)
- 03_ → genera_fig_secondo_punto.py (catena separata, mai tocca 01_/02_)
- genera_fig_modello_rumore.py (isolato, nessuna dipendenza)

esegui_tutto.py le esegue comunque tutte in un ordine fisso (01_, 02_, 03_, poi validazioni, poi tutte le figure in ordine alfabetico) — più semplice da leggere nel log, anche se non tutte le dipendenze lo richiederebbero.

Tutti i comandi, in un unico blocco (nell'ordine di esegui_tutto.py)

```
cd pacchetto_dimero_rumoroso_riproducibile
python3 01_genera_dati_vqe_ideale.py
python3 02_genera_dati_vqe_noise_aware.py
python3 03_genera_dati_secondo_punto.py
cp dati/*.npz codice/
cd codice
python3 validate_correlatori_rumorosi_dimero.py
python3 validate_scan_parametri_rumore_dimero.py
python3 validate_vqe_noise_aware_dimero.py
python3 validate_correlatori_readout_asimmetrico.py
cd ..
python3 genera_figure/genera_fig_F_vs_N_confronto.py
python3 genera_figure/genera_fig_correlatore_confronto.py
python3 genera_figure/genera_fig_correlatori_rumorosi.py
python3 genera_figure/genera_fig_griglia_completa.py
python3 genera_figure/genera_fig_modello_rumore.py
python3 genera_figure/genera_fig_readout_asimmetrico.py
python3 genera_figure/genera_fig_robustezza_multistart.py
python3 genera_figure/genera_fig_scan_eps2q_confronto.py
python3 genera_figure/genera_fig_scan_parametri.py
python3 genera_figure/genera_fig_secondo_punto.py
python3 genera_figure/genera_fig_trappola_transpilazione.py
python3 genera_figure/genera_fig_trotter_rumoroso.py
python3 genera_figure/genera_fig_vqe_dm_rumoroso.py
```

Ciascuna validazione stampa un resoconto dettagliato con i valori attesi accanto a quelli calcolati, e termina con un errore (`AssertionError`) se un controllo fallisce — non un semplice messaggio ignorabile.

5 Come sapere se un numero è quello giusto

Ogni script stampa a schermo il valore appena calcolato **accanto** al valore atteso (commento # atteso: ... o riga di stampa dedicata) — non serve confrontare a memoria con i documenti.

Se un numero non coincide

Qualcosa nel tuo ambiente locale (versione di Qiskit, seed del generatore casuale, ordine di esecuzione) sta producendo un risultato diverso: fermati e confronta prima di proseguire, non ignorare lo scostamento.

6 Due insidie già trovate e corrette

Non transpilare mai il circuito intero per i passi di Trotter ripetuti

Se transpili l'intero circuito assemblato (ancilla + preparazione + N passi + misura) a un livello di ottimizzazione alto, Qiskit può riconoscere che gli N passi identici formano un unico blocco e comprimerli in un circuito a costo costante — cancellando silenziosamente la dipendenza da N che è l'oggetto stesso dello studio. La soluzione già in uso ovunque in questo pacchetto: transpilare una volta il singolo passo, poi comporre il blocco già compilato N volte senza più transpilare.

Alcune funzioni hanno il punto di lavoro fissato come costante di modulo

In particolare `vqe_energia_fedelta_rumorosa()` (in `vqe_dm_rumoroso_dimero.py`) e `vqe_noise_aware()` (in `vqe_noise_aware_dimero.py`) costruiscono internamente l'Hamiltoniana usando `b`, `D`, `J` definiti in testa al modulo (il punto "test 2"), non un argomento della funzione. Per lavorare a un punto diverso (come nello script `03_genera_dati_secondo_punto.py`) serve sovrascrivere esplicitamente quegli attributi di modulo **prima** di chiamare la funzione:

```
import vqe_noise_aware_dimero as vna_module
vna_module.b, vna_module.D, vna_module.J = -0.18, 1.0, 1.0
```

Altrimenti l'ottimizzazione avviene silenziosamente sul punto di lavoro sbagliato, senza nessun errore — solo numeri finali non corrispondenti a quelli attesi. Questo pacchetto lo fa già correttamente in `03_genera_dati_secondo_punto.py`; se scrivi un nuovo script per un terzo punto di lavoro, applica lo stesso schema.

7 Cosa NON è incluso in questo pacchetto

I documenti `.tex/.pdf` che usano queste figure (i cinque documenti della pipeline base, l'estensione VQE noise-aware, l'estensione readout asimmetrico) sono deliverable separati, già consegnati in precedenza — questo pacchetto riproduce i **numeri e le figure**, non il testo che li commenta. I nomi dei file delle figure qui prodotte coincidono esattamente con quelli richiamati nei documenti (`\includegraphics{figure/nome.pdf}`), quindi puoi copiare la cartella `figure/` accanto a un documento `.tex` per ricompilarlo.

Verificato end-to-end, non solo assemblato

L'intera pipeline è stata rieseguita da zero (cartelle `dati/` e `figure/` svuotate, poi `esegui_tutto.py` rilanciato) dopo ogni modifica sostanziale al codice, incluse quelle di commento — ogni numero riportato in questa guida è stato ricontrollato, non solo riportato dalla prima stesura.